Autómatas, Teoría de Lenguajes y Compiladores

Diseño e implementación de un lenguaje

Informe final

Teo Fumagalli

Ingeniería Informática, ITBA
Buenos Aires, Argentina
13 de noviembre de 2026

Índice

1. Introducción **2**

2. Modelo Computacional **2**

2.1. Dominio 2

2.2. Lenguaje 2

2.2.1. Construcciones básicas 2

2.2.2. Tipos de datos 4

2.2.3. Variables y constantes 7

2.2.4. Funciones y componentes 7

2.2.5. Composición de funciones 8

2.2.6. Recursos externos 9

2.2.7. Módulos 9

3. Implementación **9**

3.1. Frontend 10

3.1.1. Scanner y Parser 10

3.1.2. Construcción del AST 11

3.2. Backend 11

3.2.1. Resolución de dependencias 11

3.2.2. Tabla de Símbolos 12

3.2.3. Análisis Estático y Typechecking 12

3.2.4. Evaluación 13

3.2.5. Generación de Código 14

3.3. Adicionales 14

3.3.1. Funciones *builtin* 14

3.3.2. Recursos externos 15

3.3.3. Importación de archivos CSV 15

3.4. Dificultades encontradas 16

3.4.1. Ambigüedad gramática en expresiones 17

3.4.2. Resolución transitiva de símbolos importados 17

3.4.3. Otras dificultades 18

4. Futuras Extensiones **18**

5. Conclusiones **19**

1. Introducción

2. Modelo Computacional

2.1. Dominio

Frog es un lenguaje de *templating* para producir sitios web estáticos en HTML que puedan ser hosteados por un servidor web estándar (e.g. Apache, nginx). Combina la sintaxis sencilla de un lenguaje de *markup* para definir elementos de un modo menos verborrágico que HTML con la potencia de un lenguaje de programación, permite crear componentes reutilizables, e integrar en los programas algunas tareas de procesamiento de datos.

Presenta una alternativa minimalista para escribir sitios web estáticos, con un *toolchain* simple, sin dependencias externas, y sin requisitos en *runtime* más que un servidor HTTP común.

2.2. Lenguaje

A continuación se detalla la especificación básica del lenguaje *frog*.

2.2.1. Construcciones básicas

Frog es un lenguaje **basado en expresiones**, es decir, la mayoría de construcciones en el lenguaje son expresiones que resuelven o evalúan a un valor. Las expresiones pueden contener otras expresiones, hasta llegar a **primitivas**. Un programa está compuesto por expresiones, y el programa entero se resuelve a un valor.

Por ejemplo, el siguiente programa genera el texto "Hola TLA!":

```
"Hola TLA!"
```

Un programa algo más complejo retorna una lista con los primeros 10 elementos de la secuencia fibonacci:

```
current := 1
last := 0

fib := for x in 0..10 {
  n := current
  current = n + last
  last = n
  n
```

```
}

fib
```

Y el siguiente programa hace lo mismo, pero mediante una función recursiva:

```
fib is function(n: int) -> int {
  if n < 0 {
    0
  } else if n < 2 {
    1
  } else {
    fib(n - 1) + fib(n - 2)
  }
}

for n in 0..10 { fib(n) }
```

Algunas expresiones son las típicas encontradas en un lenguaje de programación: literales, variables, operadores binarios, etc. Otras tienen comportamiento especial, o son expresiones que en un lenguaje imperativo convencional no lo serían (por ejemplo, control de flujo):

1. Una asignación retorna el valor asignado a la variable; esto permite encadenar asignaciones. Las asignaciones tienen comportamiento especial.
2. Una llamada a función retorna el valor de retorno de la función.
3. Una expresión condicional retorna el valor de la rama que haya sido tomada en base a la o las condiciones.
4. Un *loop* retorna una lista de tipo `T[]`, donde `T` es el tipo que retorna la expresión interna.

Un bloque de código, delimitado por `{}`, también es una expresión, que consiste de un agrupamiento lineal y ordenado de otras construcciones. Los bloques son la principal forma de organizar código en *frog*. El valor de retorno de un bloque está dado por las siguientes reglas:

1. Toda expresión dentro del bloque, **excepto las asignaciones**, agrega su valor de retorno al del bloque.
2. Si el bloque contiene una única expresión que no es una asignación, retorna directamente su valor.

3. Si el bloque contiene varias expresiones, entonces retorna una tupla con los valores de todas sus expresiones (excepto asignaciones).

4. Un bloque que contiene únicamente asignaciones, o construcciones sin valor propio (detalladas más adelante) no tiene valor de retorno. En la mayoría de contextos, esto es un error.

Además de ser una expresión que retorna un valor, un bloque define un *scope* léxico, que delimita el alcance de las declaraciones presentes dentro del mismo.

Las construcciones que no resuelven a un valor, sino que modifican el estado del programa, son:

- Declaraciones de variables o constantes
- Definiciones de tipo
- Definiciones de función
- Directivas

Estas construcciones se detallan más adelante.

2.2.2. Tipos de datos

El lenguaje utiliza un **sistema de tipos estructural**, es decir, los tipos se comparan por su estructura y no su identidad. Un objeto tipado (variable, parámetro de función, etc.) acepta un valor de cualquier tipo compatible.

Se definen los siguientes tipos de datos primitivos:

- `int`, números enteros con signo de 64 bits.
- `float`, números decimales en representación IEEE 754 de doble precisión.
- `bool`, toma los valores `true` o `false`.
- `string`, cadenas de texto.
- `element`, elementos html con un tag, atributos, y elementos hijos.
- `range`, intervalo entre dos enteros. Expresado únicamente en literales para iteración y *array slicing*.
- `nil`, el tipo del valor nulo. Generalmente utilizado en una unión para representar tipos opcionales.

Además, el lenguaje define los siguientes tipos compuestos, que pueden utilizarse para construir tipos de datos más complejos:

- Listas (`T[]`), una lista con elementos de tipo `T` y tamaño arbitrario.
- Tuplas (`[T, U, V, ...]`), una tupla con un número conocido de elementos que pueden tener distintos tipos, definidos por su posición.
- Vectores (`[T; n]` donde `n: int`), equivalentes a una tupla donde todos los valores tienen el mismo tipo, o una lista de tamaño estáticamente conocido.
- Mapas (`V[K]`), un mapa de elementos de tipo `V` indexados por llaves de tipo `K`.
- Structs (`struct`), una estructura con miembros nombrados, cada uno con un tipo definido.
- Uniones (`union`), representan un tipo que acepta cualquiera de sus miembros.

El sistema de tipado utiliza los conceptos de **igualdad**, **aceptación** y **compatibilidad** de tipos.

Igualdad Los tipos `T`, `U` son **iguales** si y sólo si tienen exactamente la misma estructura en todos sus niveles.

Aceptación Un tipo `T` **acepta** a otro tipo `U` si cumple ciertas condiciones, dependiendo de la forma de `T`:

- Un tipo primitivo se acepta únicamente a sí mismo, con la excepción de `float` que también acepta `int`.
- Una lista `T[]` acepta
  1. Otra lista `U[]`, si y sólo si `T` acepta `U`.
  2. Un vector `[U; n]`, si y sólo si `T` acepta `U`.
  3. Una tupla `[U1, ..., Un]`, si y sólo si `T` acepta `Ui` para todo `1 <= i <= n`.
- Un vector `[T; n]` acepta
  1. Otro vector `[U; m]`, si y sólo si `m == n` y `T` acepta `U`.
  2. Una tupla `[U1, ..., Um]`, si y sólo si `m == n` y `T` acepta `Ui` para todo `1 <= i <= m`.
- Una tupla `[T1, ..., Tn]` acepta

1. Un vector `[U; m]`, si y sólo si `m == n` y `Ti` acepta `U` para todo `1 <= i <= n`.
2. Otra tupla `[U1, ..., Um]`, si y sólo si `m == n` y `Ti` acepta `Ui` para todo `1 <= i <= n`.

- Un mapa `V[K]` acepta
  1. Otro mapa `V'[K']`, si y sólo si `K' == K` y `V` acepta `V'`.
  2. Si `K == string`, un struct `S`, siempre y cuando cada uno de sus miembros sea aceptado por `V`.
- Un struct `S` acepta otro struct `R`, si y sólo si para todo miembro `s_i: Ti` de `S`, existe un miembro `r_i: Ui` de `R` tal que `Ti` acepta `Ui`.
- Una unión `S = union{T1, ..., Tn}` acepta
  1. Cualquiera de sus miembros `T1, ..., Tn`.
  2. Un tipo `U` tal que algún miembro `Ti` acepta `U`.
  3. Otra unión `R`, que cumple: para todo miembro `Ui` de `R`, `S` acepta `Ui`.

Compatibilidad Los tipos `T`, `U` son **compatibles** si y sólo si `T` acepta `U` o `U` acepta `T`. Esta definición se utiliza para determinar si dos objetos pueden ser comparados; por ejemplo, `5 == 5.0` por más que tengan tipos distintos (`int` y `float`).

En la mayoría de casos, se verifica la aceptación: al asignar a una variable o parámetro de función, o al verificar que el cuerpo de una función cumpla con el contrato definido en su prototipo (tipo de retorno). El lenguaje también hace **inferencia de tipos**, permitiendo omitir anotaciones de tipos donde los mismos se pueden determinar de las expresiones, con excepción de los tipos de retorno de función que son siempre requeridos.

Un tipo puede ser nombrado mediante un **alias** o **definición de tipo**, de la forma `TypeName is T`. Estos nombres se resuelven al momento de comparar tipos. Por ejemplo, para nombrar un struct:

```
Person is struct {
  first_name: string,
  last_name: string
}

Book is struct {
  isbn: string,
  title: string,
  author: Person
}
```

Luego, dentro del *scope* que contiene esta declaración, la expresión de tipo `Book` es equivalente a `struct { isbn: string, title: string, author: struct { first_name: string, last_name: string } }`.

2.2.3. Variables y constantes

Frog permite declarar variables y constantes dentro de un *scope*, como cualquier lenguaje imperativo. Una declaración de variable tiene la forma `name: [Type] = <expression>`, donde el tipo puede omitirse y se infiere del valor inicial. Toda declaración requiere inicializar la variable con un valor inicial, no está permitido declarar una variable sin valor.

Una variable existente en el *scope* actual (independiente de dónde haya sido definida) puede ser reasignada con una expresión de asignación `name = <expression>`.

Una constante se declara de modo análogo a una variable, con una declaración de constante de la forma `name:: [Type] = <expression>`. Funciona de modo idéntico a una variable, con la diferencia que no se permite reasignarla. Reasignar una constante genera un error de compilación.

2.2.4. Funciones y componentes

El lenguaje permite definir *funciones* para reutilizar código, de forma similar a cualquier lenguaje imperativo. Las funciones en *frog* tienen algunas características particulares:

- Las funciones actúan como un punto de corte en la resolución de *scope* para variables. Es decir, dentro de una función, no se permite acceder variables externas a la misma.
- Los parámetros de función no pueden ser modificados por la misma, dentro del *scope* de la función están declarados como constantes.
- Las funciones deben tener un valor de retorno.

Estas características implican que las funciones en *frog* son puras, es decir, no tienen efectos secundarios fuera de retornar un valor en base a sus parámetros.

Las funciones se definen de modo similar a un tipo de datos, con la adición de un cuerpo de función que contiene su código:

```
display_name is function(user: User) -> string {
  concat(" ", user.first_name, user.last_name)
}
```

La sintaxis de llamada de función es similar a un lenguaje como C, con algunas características adicionales:

Los parámetros de función pueden ser opcionales (si su tipo acepta `nil`) o tener definido un valor por defecto, en cuyo caso pueden ser omitidos de la llamada:

```
with_defaults is function(x: int, y: int?, z: int = 5) -> string {
  ...
}

with_defaults(1, 2, 3)
with_defaults(1)
```

Los argumentos en una llamada a función pueden especificar un nombre de parámetro con la sintaxis `name = <expression>`. Los argumentos con nombre deben ir después de todos los argumentos posicionales.

```
with_defaults(x = 1, z = 3)
with_defaults(1, z = 3)
with_defaults(z = 3, x = 1, y = 2)
```

Una función que toma parámetros con nombre `class` o `id` soporta una sintaxis de llamada especial, donde estos parámetros pueden pasarse en formato similar a un selector de CSS:

```
div.flex.flex-row.gap-4.items-center#myDiv()

// Equivalente a

div(class = ["flex", "flex-row", "gap-4", "items-center"], id = "myDiv")
```

Se llama **componentes** a las funciones usadas dentro de una estructura que genera HTML. Generalmente retornan `element` y pueden tomar composición.

2.2.5. Composición de funciones

La construcción que permite utilizar el lenguaje para construir estructuras HTML complejas de modo sencillo es la **composición de funciones**. Esto consiste en pasar un valor adicional como parte de una llamada a función. La expresión es tratada como un parámetro especial, y es inserta en cada lugar donde la función tenga la palabra reservada `compose`.

Una función acepta composición si se declara con un `tipo de ocmposición`:

```
twice is function compose string -> [string; 2] {
  compose
  compose
```

```
}

foo := twice "Hello"    // ["Hello", "Hello"]
bar := twice {
  who := "world"
  "Hello ${who}!"
}                       // ["Hello, world!", "Hello, world!"]
```

Un bloque de código pasado para composición captura el *scope* de la llamada a función; es decir, tiene acceso a las variables del lugar donde se llama. **No tiene acceso al *scope* interno de la función**, ya que se evalúa fuera de la llamada.

2.2.6. Recursos externos

2.2.7. Módulos

3. Implementación

El compilador fue implementado en C, utilizando Flex + Bison para el frontend, y una arquitectura separada en etapas bien definidas diseñada para cumplir los requisitos de la especificación:

- El compilador debe poder trabajar con varios archivos de entrada
- Algunos archivos de entrada producen una salida, otros no
- Permite importar símbolos de otros archivos, si fueron declarados

Para cumplir con estas condiciones, fueron definidas las etapas del compilador:

1. Parsing y construcción del AST
2. Resolución de dependencias
3. Construcción de la tabla de símbolos y verificación de símbolos estáticos
4. Análisis estático y typechecking
5. Evaluación
6. Generación de código

Las etapas se ejecutan en orden, cada una sobre todos los archivos de entrada en un orden determinado (ver Subsubsección 3.2.1). Cada etapa puede reportar ciertos errores. Al terminar la etapa (sobre todos los archivos), se acumulan los errores y/o *warnings*, se muestran por salida estándar (`stderr`) y de haber errores graves se frena la compilación.

Cada etapa tiene su propio módulo en el compilador:

- **Frontend:** Código de Flex, Bison, funciones de soporte y `frontend`
- **Análisis estático**: `backend/static_analysis`
- **Evaluación**: `backend/eval` y `backend/builtins`
- **Generación de código**: `backend/codegen`

Además, ciertos módulos son utilizados por varias etapas del compilador:

- `ast/ast` define las estructuras del AST y sus constructores. Usado por todas las etapas entre parsing y evaluación.
- `ast/symbols` define las estructuras de la tabla de símbolos, usadas por las etapas de análisis estático y evaluación.
- `error` se encarga del manejo de errores, exponiendo funciones para reportar y contar/verificar errores (*error gate*).
- `support/state` define el estado global del compilador, usado en todas las etapas.

Otros módulos en `support` exponen funciones útiles y estructuras de datos (array dinámico, hash map) usados en distintos lugares del compilador.

3.1. Frontend

La etapa de parsing y construcción del AST involucra todo el frontend del compilador. Para cada archivo, produce un AST. Puede generar errores de sintaxis, y algunos otros detectables en esta etapa (por ejemplo, importar un archivo inexistente).

Esta etapa se ejecuta en el orden en el que fueron especificados los archivos de entrada por línea de comandos, ya que es previa a la resolución de dependencias.

3.1.1. Scanner y Parser

El scanner y parser utilizan Flex + Bison en modo *push*. Las acciones de Flex son relativamente mínimas: generan tokens, en algunos casos les asignan metadata relevante, e interpretan literales numéricas y booleanas.

Las acciones de Bison, junto con la definición de la gramática, forman el núcleo de esta etapa: construyen el AST de forma ascendente (correspondiendo al orden del parser LALR de Bison) llamando los constructores del módulo `ast`, y resuelven algunas construcciones puramente sintácticas (por ejemplo, la transformación de un tipo opcional `T?` en la interpretación interna `union{T, nil}`).

3.1.2. Construcción del AST

El parser llama a las funciones del módulo AST que construyen los distintos objetos. De no tener errores, el resultado es un AST válido que siempre tiene como raíz un `Program`.

El AST de cada archivo se almacena en el estado del compilador para ser utilizado por el backend.

3.2. Backend

El backend del compilador incluye todas las otras etapas, desde resolución de dependencias y análisis estático hasta generación de código. A continuación se detalla cada una.

Las etapas del backend son ejecutadas sobre cada archivo en un orden definido por el grafo de dependencias, que se calcula en la primera parte.

3.2.1. Resolución de dependencias

Esta es una pseudo-etapa que se ejecuta entre el frontend y backend del compilador. Una de las funcionalidades del lenguaje es el uso de las directivas `@export` y `@import` para exportar símbolos estáticos (tipos y funciones) de un archivo y utilizarlos en otros. Esto genera dependencias entre estos archivos, y por lo tanto el orden de análisis y evaluación debe seguir el orden de estas dependencias.

El grafo de dependencias (que, para un programa bien formado, es un digrafo acíclico o DAG) es formado en el frontend durante la construcción del AST, que al procesar una directiva `@import` marca al archivo que la contiene como dependiente del archivo de origen.

Inmediatamente seguido a la etapa frontend (y suponiendo que termina sin errores), se analiza este grafo de dependencias para definir el orden en el que deben ser evaluados los archivos, teniendo en cuenta que cada archivo debe ser evaluado luego de los archivos de los que dependa. Este proceso también detecta errores como dependencias circulares, en cuyo caso emite un error y detiene la compilación.

3.2.2. Tabla de Símbolos

Esta etapa cumple cuatro funciones principales:

1. Completa la tabla de símbolos estáticos: definiciones de tipo y función.
2. Detecta errores de redefinición de tipos o funciones, y algunos casos de uso de tipos o funciones no definidos.
3. Calcula el estado del *stack* en cada *scope*, y lo asocia al AST en los lugares relevantes (expressiones *block*, funciones y tipos).
4. Resuelve las directivas `@import`, definiendo en el *scope* local los símbolos importados y validando su existencia, y `@export`, marcando como exportados los símbolos.

Para esto, recorre el AST de cada archivo en orden, armando el *stack* en el camino y resolviendo las definiciones de tipo y función.

3.2.3. Análisis Estático y Typechecking

Esta etapa es la más compleja en el compilador, y se encarga de asegurar la correctitud del programa tanto en cuanto al sistema de tipos, como a otros aspectos detectables estáticamente: validación de referencias, llamadas a función, etc.

La separación entre esta etapa y las siguientes (evaluación y generación de código) es tal que cumpla con las siguientes condiciones:

1. Las etapas posteriores pueden asumir que el programa es correcto; las únicas verificaciones que deben hacer son las que corresponden a su ejecución (por ejemplo, índices a una lista *out of bounds*, en el caso en que el tamaño de la lista no es conocido por su tipo).
2. Una implementación alternativa (por ejemplo, que genere un ejecutable o bytecode para una VM que corra en un servidor para generar páginas a demanda mediante SSR, en lugar de HTML estático) sólo debe reemplazar las etapas posteriores, y no debe realizar ningún análisis estático adicional.

En esta etapa se hace un segundo recorrido por el AST de cada archivo, verificando que todas las expresiones cumplan con las invariantes que define el sistema de tipos del lenguaje y que sean válidas en otros aspectos. A grandes rasgos la implementación se separa en cuatro partes:

Definición y resolución de símbolos Funciones que obtienen símbolos de la tabla, siguiendo las reglas de *scoping* del lenguaje, y definen nuevos símbolos (variables).

Inferencia de tipos Funciones que retornan, para una expresión, el tipo de datos que la representa. Se llaman recursivamente.

Verificación de tipos Verifican que se cumplan invariantes del sistema de tipos en las expresiones. Generalmente implementado dentro de las funciones de inferencia.

Comparación de tipos Funciones que determinan si dos tipos son iguales, o, dado el sistema de tipado estructural del lenguaje, si son compatibles ("A entra en B"). Se encargan de resolver referencias a tipos con el *scope* correcto, incluso a través de distintos archivos. Usadas por la verificación de tipos.

3.2.4. Evaluación

La etapa de evaluación es el *interpreter* integrado en el compilador: se encarga de transformar cada AST en un único valor que luego puede ser interpretado y transformado en HTML por el generador de código. Al ser posterior al análisis estático, esta etapa asume que todo programa que recibe es válido, y no hace verificaciones adicionales excepto aquellas que no pueden resolverse estáticamente (por ejemplo, acceso a índices *out of bounds* en listas, o leer de un mapa un *key* no definido).

Como no debe hacer verificación de tipos, el evaluador hace una forma de *type erasure*: los tipos de valor generados en la evaluación no llevan directamente información de tipo del lenguaje, ya que no es de particular relevancia para la evaluación si se asume que todas las invariantes de tipado se cumplen. Los tipos de valor se corresponden generalmente a uno o más tipos de datos del lenguaje:

Tipo de valor	**Implementación**	**Tipos de datos**
nil	Valor nulo	nil
int	Enteros de 64 bits	int
float	IEEE 754 doble precisión (64 bits)	float
string	Cadenas de texto	string
bool	Valores booleanos	bool
element	Definición de elementos HTML	element
list	Listas ordenadas de valores	list, vector, tuple
map	Hash maps	map, struct
range	Intervalos entre enteros	range

Cuadro 1: Tipos de valor utilizados por el evaluador

En particular, los valores `list` y `map` contienen valores genéricos, sin restricciones sobre su tipo; se asume que, por ejemplo, una lista que estaba tipada como vector de enteros contiene únicamente enteros, ya que este es el caso para todo programa válido.

Si bien esta etapa podría estar integrada con el análisis estático, dado que el compilador interpreta el programa y produce como salida código HTML, la implementación la mantiene separada por dos razones: primero, para mantener el principio de única responsabilidad; segundo y principalmente, para facilitar una conversión a un backend alternativo que produzca, en lugar de un sitio estático, código usable por algun *runtime* para generar páginas a demanda con información de cada pedido HTTP (SSR).

En la etapa de evaluación se recorre nuevamente el AST, esta vez colapsando cada expresión recursivamente en su valor correspondiente. Por ejemplo, operaciones binarias se reemplazan con su resultado, llamadas a función con su valor de retorno, etc. La implementación de esta etapa es relativamente sencilla, y consiste principalmente en manejar todos los casos.

3.2.5. Generación de Código

La etapa de generación de código se encarga de transformar los valores producidos por el evaluador en código HTML y emitirlo a un archivo. La implementación es sencilla, y simplemente recorre un valor entrando de modo recursivo en listas, mapas y descendientes de elementos HTML y transformándolos en una cadena de texto que corresponde a HTML válido.

En el caso de tipos de valor que no corresponden directamente a HTML, se convierten en una representación de texto. La excepción son las listas, que simplemente emiten todos sus elementos en secuencia.

3.3. Adicionales

3.3.1. Funciones *builtin*

El lenguaje cuenta con funciones *builtin*, que hacen de base para muchas de las operaciones que permite. Estas funciones están implementadas directamente en el compilador, de modo que toman como parámetros la lista de argumentos al momento de evaluarse, la referencia a la función, entre otras cosas. Se definen y se registran como símbolos como parte del módulo `backend/builtins`.

Las más notables son la familia de funciones que representan todos los elementos HTML, y son la única forma válida de obtener valores de tipo `element` en el lenguaje. Estas funciones están definidas de modo tal que aceptan como parámetros todos los atributos válidos para su elemento dado (definido en una tabla en `builtins.c`), con tipos apropiados para el atributo:

Atributos	Tipos de datos
boolean attributes	`bool?`
`dataset`, `aria`	`attr_type[string]?`
Otros atributos	`attr_type?`

Donde `attr_type` es equivalente a `union{int, float, string, bool, union{int, float, string, bool}[]}`.

Además, todas las funciones que representen elementos no *void* (TODO ref MDN) toman como tipo de composición cualquier tipo primitivo o elemento HTML, o una lista de estos tipos. Todas las funciones HTML retornan un valor de tipo `element`.

3.3.2. Recursos externos

El lenguaje permite incluir recursos externos en el sitio de distintos modos, utilizando la expresión `import`. Esta expresión importa un archivo externo al código y lo interpreta dependiendo del modo:

Modo	Interpretación	Tipo de retorno
`url`	Transforma el path al archivo en una URL relativa, utilizando las opciones del compilador	`string`
`raw`	Lee los contenidos del archivo verbatim como cadena de texto. Asume archivo de texto.	`string`
`csv`	Lee un archivo CSV e interpreta los datos; ver Subsubsección 3.3.3	varía

3.3.3. Importación de archivos CSV

Uno de los recursos externos que se pueden incluir en una página es un CSV como fuente de datos. La expresión `import csv` parsea el CSV, utilizando un buffer secundario en el parser que modifica el contexto de Flex para reconocer tokens de CSV. La gramática de Bison luego incluye como subset, utilizando estos tokens, la gramática de CSV según (TODO CSV RFC ref). Los valores se guardan como metadatos en el nodo que representa la expresión en el AST.

La expresión retorna una tupla `[string[], row_type[]]`, donde el primer componente es una lista de *headers* (los valores en la primera fila del CSV) y el segundo es una lista de cada fila que sigue a la primera, interpretada según el tipo `row_type`.

Esta expresión tiene la particularidad de que puede ser tipada como `import csv of T`, en cuyo caso `row_type is T`. De omitirse el tipo, `row_type` toma su valor por defecto `string[string]`, donde cada fila es un mapa indexado por los valores de *headers*.

Un tipo para esta expresión debe ser *compatible con CSV*, lo cual significa que debe ser uno de:

- Un vector de tamaño igual a la cantidad de columnas, cuyo tipo interno es un tipo de campo CSV válido.
- Una tupla de tamaño igual a la cantidad de columnas, cuyos tipos son todos tipos de campo CSV válidos.
- Una lista cuyo tipo interno es un tipo de campo CSV válido.
- Un mapa con tipo de clave `string` y tipo de valor que corresponde a un tipo de campo CSV válido.
- Un struct, cuyos miembros coinciden exactamente en nombre con los *headers* del CSV, y tienen tipos de campo CSV válidos.
- Una unión en la cual al menos un miembro cumple alguna de las anteriores. En caso de que varios miembros cumplan, se toma el primero por orden de definición.

Un tipo es válido como *tipo de campo CSV* si es:

- Un tipo primitivo `string`, `int`, `float` o `bool`
- Un alias para un tipo de campo CSV válido
- Una unión que contiene al menos un tipo que cumple las anteriores. En caso de haber varios, se prioriza `string` al ser el menos restrictivo.

En momento de evaluación, el evaluador intenta transformar los valores del CSV al tipo de fila dado. En el caso de tipos lista (lista, vector o tupla) se transforma cada campo al tipo correspondiente a su posición y se ubican en orden de columna (izquierda a derecha). En el caso de tipos mapa (mapa o struct), se transforma cada campo al tipo correspondiente y se ubican en la fila indexados por el *header* corresopndiente a su columna. De no poder convertir un valor al tipo solicitado, se produce un error.

3.4. Dificultades encontradas

Se encontraron numerosas dificultades a lo largo de la implementación del lenguaje, algunas que necesitaron cambios a la especificación, otras de menor consecuencia. Se listan a continuación; las más importantes tienen su propia sección y las menores son agrupadas en una sección al final.

3.4.1. Ambigüedad gramática en expresiones

La especificación original del lenguaje ignoraba todos los caracteres *whitespace*, incluyendo saltos de línea, implícitamente permitiendo múltiples expresiones en una línea. Durante el desarrollo del frontend, se encontraron problemas de ambigüedad gramática difíciles de resolver con este funcionamiento, debido a la falta de un delimitador entre expresiones. Por ejemplo, `foo[2]` podía ser interpretado como un acceso a lista por índice (tercer elemento de `foo`), o como una expresión que retorna el valor de `foo` continuada por una literal de lista de un elemento.

Se consideraron varias opciones, y se decidió introducir un separador de expresiones utilizando el salto de línea para esta función. Esto permitió distinguir expresiones explícitamente. En algunos casos particulares se permiten saltos de línea dentro de una expresión (listas, structs, etc.), pero están controlados de modo de evitar la ambigüedad.

3.4.2. Resolución transitiva de símbolos importados

Durante la implementación del sistema de exportación e importación de símbolos entre archivos, se encontraron varias dificultades con las dependencias transitivas, es decir, cuando un símbolo exportado depende de otros que no fueron exportados. Algunos ejemplos son:

- Un archivo exporta una función cuyo tipo de retorno, o el tipo de uno de sus parámetros, es un alias definido en el archivo pero no exportado.
- Un archivo exporta un alias de un tipo compuesto, que contiene un alias a otro tipo que no fue exportado.
- El tipo de retorno o de un parámetro de una función exportada es un tipo compuesto y contiene un alias no exportado.

Estos casos también expusieron problemas con la implementación sencilla de resolución de tipos (utilizando siempre el *scope* local donde fueron referenciados), por ejemplo, la definición de un tipo compuesto que contiene un alias cambiando si se redefine dicho alias en algún *scope* interior (*shadowing*).

Resolverlo llevó a rediseñar múltiples veces el sistema de manejo de scopes y el stack, llevando a la solución actual que involucra una variante limitada de *lexical scoping*, donde cada bloque de código, tipo, y definición de función contiene una referencia a su scope incluyendo el stack completo en el lugar de su definición, y este stack es utilizado para resolver símbolos dentro de su contexto.

Como efecto secundario, esta implementación hace relativamente sencillo agregar captura de scope a las funciones, permitiendo pasar funciones como variables, una funcionalidad

que fue dejada fuera de la especificación original justamente por su dificultad de implementación. Si bien no fue implementado por cuestiones de tiempo y por entrar en conflicto con otros aspectos del lenguaje, podría agregarse en una versión a futuro.

3.4.3. Otras dificultades

- Los tipos de enumeración (`enum`) fueron eliminados del lenguaje, ya que son difíciles de hacer funcionar con tipado estructural (efectivamente requieren un tipo con nombre) y no se los consideró esenciales. Opciones de reemplazo a futuro podrían ser uniones de tipos literales, un patrón común en lenguajes como Typescript.
- Al usar uniones como tipo de fila CSV, idealmente se tomaría el tipo menos restrictivo. Se omitió esto del lenguaje, principalmente por la dificultad de definir "menos restrictivo" y falta de tiempo.
- Las operaciones aritméticas vectoriales (por ejemplo, sumar vectores) necesitaron de manejar varios casos borde, en particular en cuanto a la conversión entre `float` e `int`, y casos como una tupla mixta que contiene ambos tipos (que, si bien no es un vector, es *compatible* con un vector de `float` según las reglas del lenguaje).
- Determinar cuándo y dónde evaluar los argumentos por defecto de una función presentó dificultades relacionadas al *scoping*, y la solución alcanzada no es particularmente ideal ya que los evalúa en cada llamada a la función cuando en realidad son siempre iguales.
- Definir el tipado de la composición de funciones vino con dificultades: la versión original según la especificación infería el tipo de la composición según el retorno de la función, pero esto va en dirección opuesta al funcionamiento normal del typecheck, que primero infiere el tipo de una expresión (en este caso, el cuerpo de una función) para luego compararlo con el tipo de retorno. Se resolvió requerir un tipo de composición explícito en las funciones que aceptan composición.

4. Futuras Extensiones

Durante el desarrollo se identificaron varias posibilidades de extensión a futuro. Algunas son extensiones a la especificación del lenguaje, otras son partes de la misma que no se implementaron, sea por tiempo o porque quedaron en conflicto con otros cambios al lenguaje realizados durante la implementación.

- Soporte para otros formatos de recursos externos, como JSON, Markdown o mismo HTML (permitiendo operar sobre documentos existentes).

- Más opciones de organización de proyecto, como algún sistema de *templates* por tipo de archivo. Si bien estaba en la especificación original, no estaba del todo definido y se decidió dejarlo fuera de la implementación.
- Mejores opciones de manipulación de strings, como por ejemplo la posibilidad de usar la sintaxis de *slicing* de listas sobre strings, o alguna forma de interpolación.
- Una expresión de decomposición de listas que permita concatenación fácil, por ejemplo `list := [head, ...tail]` o `list := [...foo, 5, ...bar]` (una lista compuesta por todos los elementos de `foo`, seguidos de 5, seguido de todos los elementos de `bar`).
- Una librería más completa de funciones *builtin*, incluyendo por ejemplo funciones aritméticas, manipulación de listas, mapas y strings, etc.
- Expresiones `switch` con pattern matching.
- Alguna forma de discriminar el tipo que contiene una unión, similar a *type guards* de Typescript (TODO ref) o `if let` en Rust(TODO ref).
- Mejor documentación, mensajes de ayuda, y manejo de algunos errores.

5. Conclusiones